

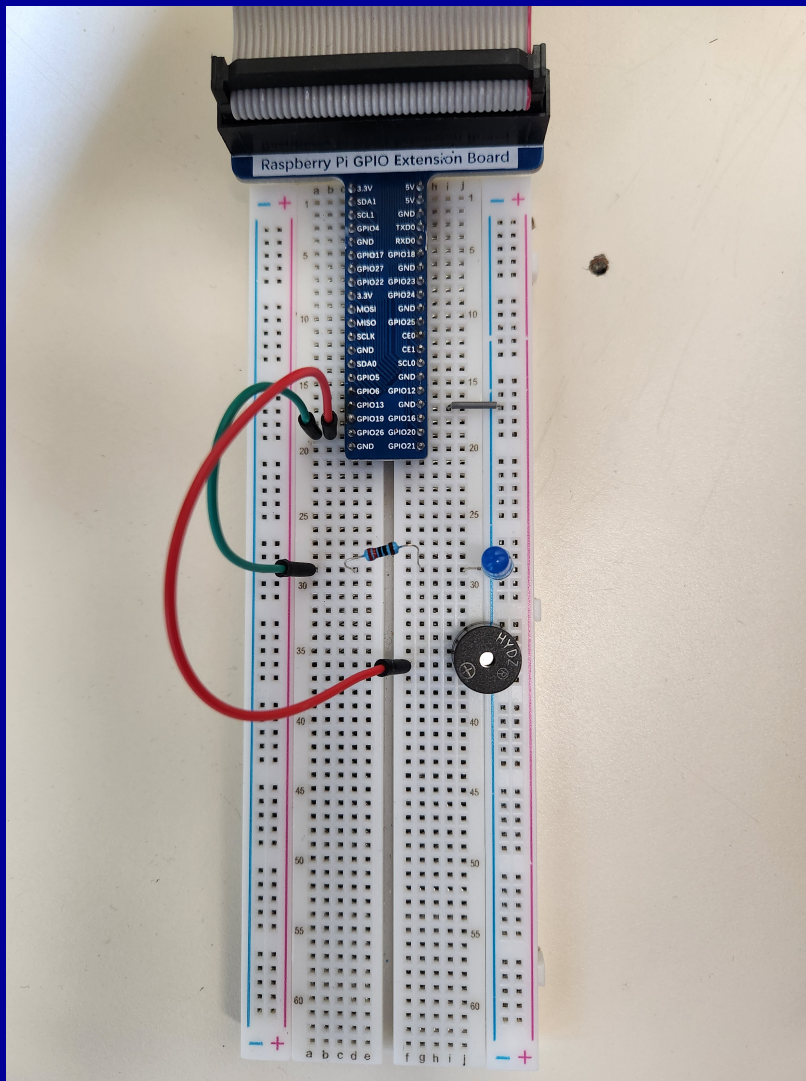
Raspberry PI

Broches en usages multiples
Lecture des entrées

Méthodes off(), close(), pause()

- off(): éteindre (tension nulle) une composante
- close(): détacher une composante d'une broche pour par exemple la réassigner à une autre composante. Ne pas le faire lèvera une exception
- pause(): mettre un script en pause pour par exemple continuer à surveiller des entrées de données
- ATTENTION: les connections multiples ne font vraiment de sens que dans l'interpréteur, puisque cela nous donne le temps de refaire les branchements nécessaires

Connections qui se chevauchent



```
chevauche1.py ✕  chevauche2.py ✕  wait_for.py ✕  bouton_LED1.py ✕

1  from gpiozero import LED, Buzzer
2  from time import sleep
3
4  led = LED(26)
5  led.on() # cela va démarrer le Buzzer aussi
6  sleep(1)
7  led.off()
8  led.close() # obligatoire pour libérer
               # la pin 26, sinon exception
10 buzzer = Buzzer(26)
11 buzzer.on()
12 sleep(1)
13 buzzer.off()
14 buzzer.close()
```

```
chevauche1.py ✕  chevauche2.py ✕

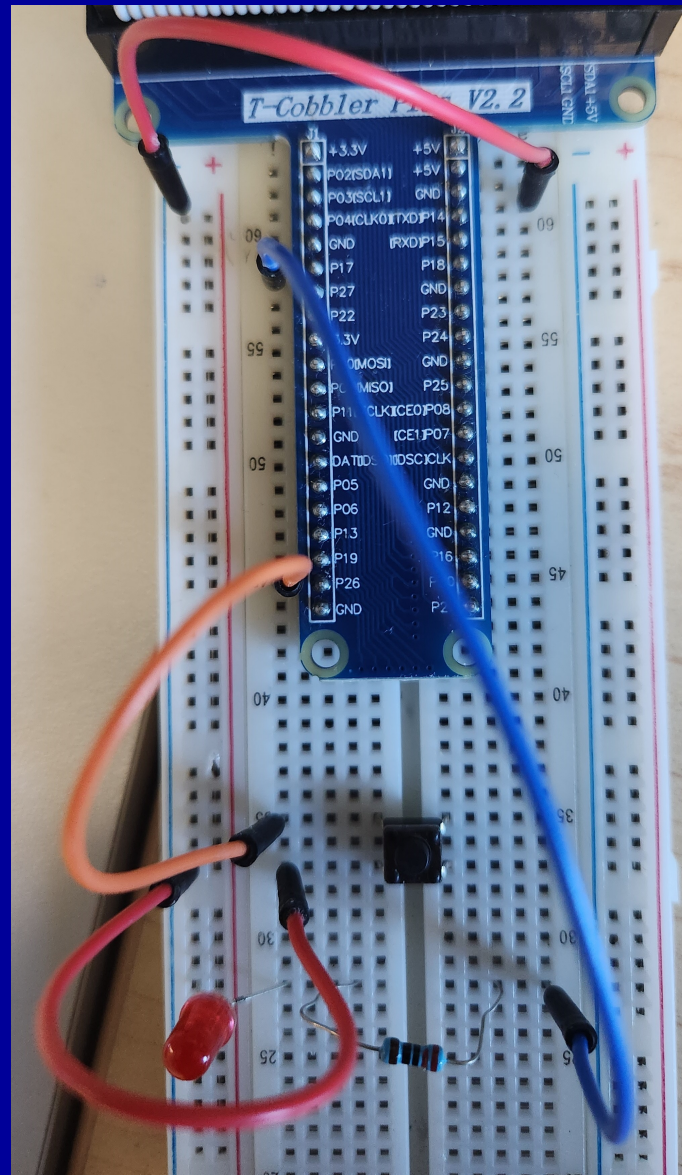
1  from gpiozero import PWMLED, Buzzer, GPIOZeroError
2  from time import sleep
3
4  try:
5      led = PWMLED(26)
6      led.value = 0.05 # il faut entre 0 et 1
7      sleep(1)
8      led.off()
9      #led.close() # permettant la réutilisation
10     bz = Buzzer(26)
11     bz.on()
12     sleep(1)
13     bz.off()
14     bz.close()
15 except GPIOZeroError as e:
16     print('EXCEPTION')
17     print(e)
```

Lectures des entrées

- Entrées
 - Button
 - MotionSensor
 - LightSensor
 - DistanceSensor
- Sorties
 - LED
 - RGBLED
 - Buzzer
 - Motor

On peut parfois lire et écrire sur des composantes (e.g. LED.value)

Approches pour gérer des entrées



Approche 1 pour un bouton: *when*

```
wait_for.py ✕ is_pressed.py ✕ when.py ✕  
1  from gpiozero import LED, Button  
2  from signal import pause  
3  
4  led = LED(17)  
5  button = Button(26)  
6  
7  def pressed():  
8      print("Pressé")  
9  def released():  
10     print("Relâché")  
11  
12  button.when_pressed = pressed  
13  button.when_released = released  
14  pause()
```

LED contrôlée par un bouton (1)

```
bouton_LED1.py bouton_LED2.py bouton_LED3.py
1 from gpiozero import LED, Button
2 from signal import pause
3 import sys
4
5 led = LED(17)
6 button = Button(26)
7 compteur = 3
8 def allumerLED():
9     global compteur
10    led.on()
11    compteur-=1
12    print("Presse # " + str(compteur))
13 def eteindreLED(l,s):
14     global compteur
15     l.off()
16     print(s + " éteint!")
17     if (compteur == 0):
18         sys.exit()
19
20 button.when_pressed = allumerLED # ou encore led.on sans ()
21 button.when_released = lambda : eteindreLED(led,"bouton ") # avec des arguments
22 # Python fonction lambda:
23 # fonction = lambda a,b:a + b
24 # print(fonction(1,2)) # affiche 3
25 print("C'est parti!")
26 pause()
```

Exercice 1: faites en sorte que le bouton se comporte comme celui d'une lampe: chaque clic change l'état de la LED

Indices: méthodes *toggle()*, propriétés *value* et *is_lit*

LED contrôlée par un bouton (2)

```
bouton_LED1.py ✕ bouton_LED2.py ✕ bouton_LED3.py ✕  
1  from gpiozero import LED, Button  
2  from signal import pause  
3  
4  led = LED(17)  
5  button = Button(26)  
6  
7  print("C'est parti!")  
8  
9  led.source = button # mis-à-jour dans un thread séparé  
10 pause()
```

LED contrôlée par un bouton (3)

```
bouton_LED1.py ✕ bouton_LED2.py ✕ bouton_LED3.py ✕  
1  from gpiozero import LED, Button  
2  from time import sleep  
3  
4  led = LED(17)  
5  button = Button(26)  
6  |  
7  print("C'est parti!")  
8  while True:  
9      led.value = button.value  
10     sleep(0.01)
```

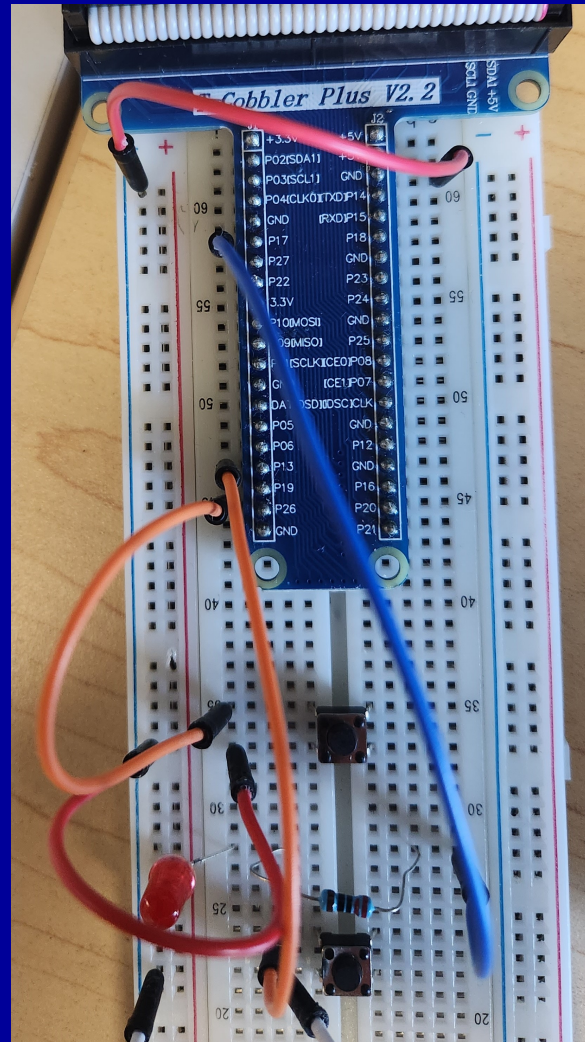
Approche 2 pour bouton: *wait*

```
wait_for.py ✕  is_pressed.py ✕  when.py ✕  
1  from gpiozero import LED, Button  
2  
3  led = LED(17)  
4  button = Button(26)  
5  
6  while True:  
7      print("Relâché.")  
8      button.wait_for_press()  
9      print("Pressé.")  
10     button.wait_for_release()
```

Approche 3 pour bouton: *is_pressed*

```
wait_for.py ✕  is_pressed.py ✕  when.py ✕  
1  from gpiozero import LED, Button  
2  from time import sleep  
3  
4  led = LED(17)  
5  button = Button(26)  
6  
7  while True:  
8      if button.is_pressed:  
9          print("Pressé")  
10         sleep(1)  
11     else:  
12         print("Relâché")  
13         sleep(1)
```

Jeu: temps de réaction (1)

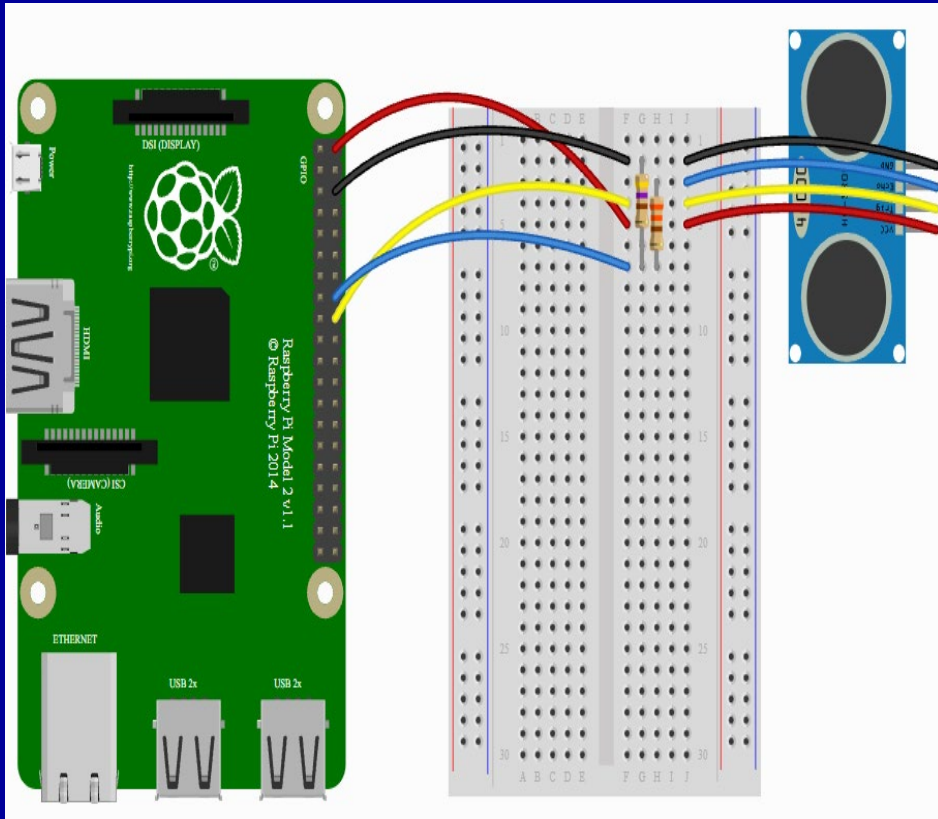


Jeu: temps de réaction (2)

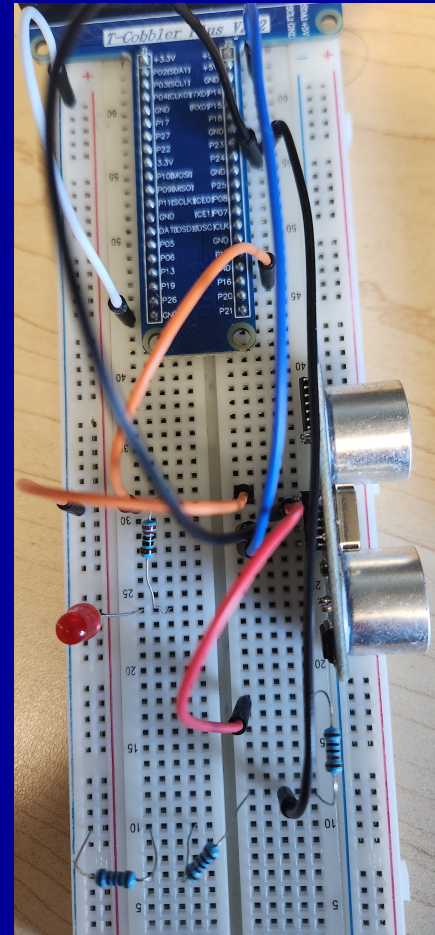
```
reaction.py ✖
1 from gpiozero import Button, LED
2 from time import sleep
3 import random
4
5 led = LED(17)
6
7 joueur_1 = Button(26)
8 joueur_2 = Button(13)
9
10 delai = random.uniform(5, 10)
11 print("C'est parti!")
12 sleep(delai)
13 led.on()
14
15 while True:
16     if joueur_1.is_pressed:
17         print("Joueur 1 gagne!")
18         break
19     if joueur_2.is_pressed:
20         print("Joueur 2 gagne!")
21         break
22
23 led.off()
```

Exercice 2: le premier joueur qui presse son bouton PENDANT le délai ne pourra gagner

Capteur de distance (1)

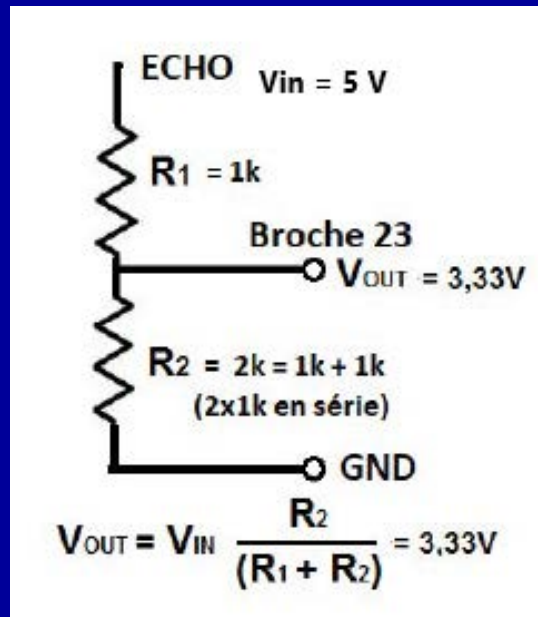


GND
ECHO
TRIG
VCC



Capteur de distance (2)

Remarquez les deux résistances: elles servent de diviseur de tension, puisque la broche ECHO du capteur fournit 5v à la broche GPIO, qui elle ne peut en prendre que 3.3v.



<https://www.digikey.ca/fr/resources/conversion-calculators/conversion-calculator-voltage-divider>

Capteur de distance (3)

```
distance1.py ✕ distance2.py ✕
1 from gpiozero import DistanceSensor
2 from time import sleep
3
4 capteur = DistanceSensor(23, 24)
5
6 while True:
7     print("Distance de l'objet le plus proche: ", capteur.distance, 'm')
8     sleep(1)
```

Exercice 3: ajouter les branchements et le code nécessaire pour que l'intensité d'une PWMLED s'ajuste en fonction de la distance de l'objet, l'intensité maximale à la distance zéro.

```
distance1.py ✕ distance2.py ✕
1 from gpiozero import DistanceSensor, LED
2 from signal import pause
3
4 capteur = DistanceSensor(23, 24, max_distance=1, threshold_distance=0.2)
5 led = LED(16)
6
7 capteur.when_in_range = led.on
8 capteur.when_out_of_range = led.off
9
10 pause()
```

Exercice 4: ajuster le code et les branchements pour transformer la LED en PWMLED et que son intensité s'ajuste en fonction de la distance de l'objet, l'intensité maximale à la distance zéro, minimale à 0.20. Quelle la différence de comportement de la LED en comparaison avec l'exemple précédent?